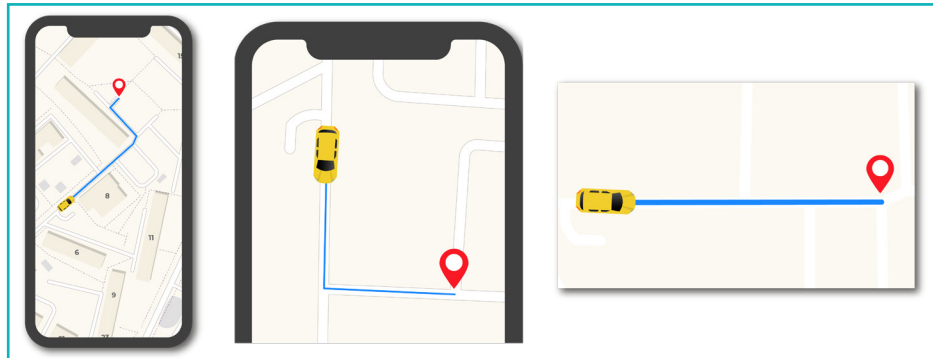




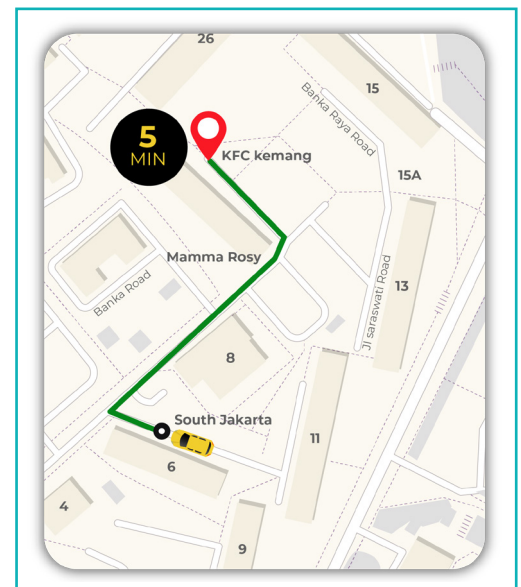
When creating Virtual Worlds, we need to simulate virtual objects and entities similar to those in the real world. Such entities can be created using the concept of 'Autonomous Agents'. Let's dive into this interesting topic and use our coding skills to explore this.



Have you ever used transportation applications like Ola or Uber? When we travel using such services, we can see a tiny vehicle symbol moving along the designated path. Do you know how it moves correctly along the path?



We can also see similar functionality in the Google Maps app when we set directions to a destination and track our movement. Such traffic simulations are more commonly found in applications like these.



Similar applications and games that incorporate traffic simulation as part of their mechanics also display such map-like visuals. For example, a game called “Cities: Skylines” is a city-building simulation game in which players take on the role of a city mayor and have the task of planning, constructing, and managing a virtual city.

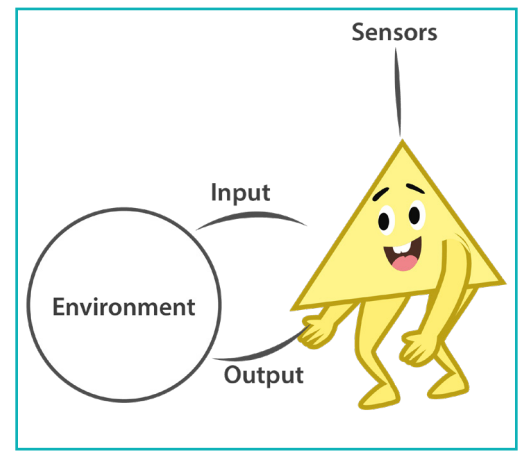


One of the critical aspects of city management in the game is Traffic simulation. In the game, we can see vehicles automatically moving on the road, including cars, trucks, buses, and emergency vehicles navigating through the city's road network based on the designed infrastructure, traffic rules, and the game's AI algorithms.

Such entities that can sense the surrounding environment and respond accordingly, simulating real-world movements in virtual worlds are called autonomous agents. Autonomous agents have a wide range of applications across different fields such as simulation and modelling, games, and virtual reality.

With the increasing prospect of AR/VR applications in daily life, the implementation of such autonomous agents with various capabilities will be very common.

Autonomous agents are virtual characters or objects with the ability to generate their own movements and actions. They do not rely on pre-programmed movements and animations; instead, they use algorithms and rules to dynamically generate actions in real-time.



Autonomous agents can be designed to perform certain tasks such as seek, arrive, follow, and many more. Such behaviours and algorithms required to implement autonomous agents are called Steering Behaviours. We will learn more about Steering behaviour algorithms later.

Here are some applications of Autonomous Agents and related algorithms for their steering behaviours.

- a. Virtual Reality** – Virtual worlds require many characters and objects that need to simulate real-world entities in the environment. The design of such entities has algorithms that create autonomous agents, at its core.
- b. Games** - They are commonly used in game development. They are used for creating realistic and intelligent non-player characters capable of pathfinding, decision-making, interacting with the game world, and providing engaging gameplay experiences.
- c. Robotics** - They play a crucial role in robotics. They are used to control and coordinate the behaviour of Autonomous robots
- d. Simulation and Modelling** - They are used in simulations and models to replicate complex systems and phenomena like traffic flow, crowd behaviour, ecological systems, etc
- e. Autonomous Vehicles** - They are core of self-driving cars and autonomous vehicle systems.

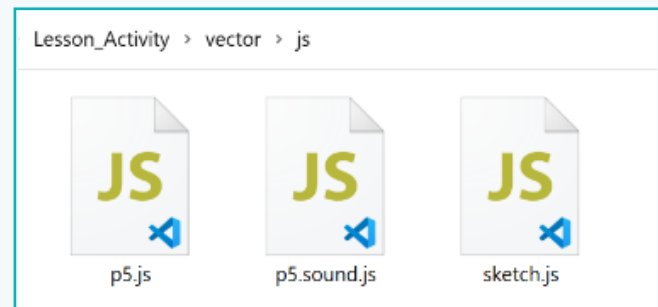
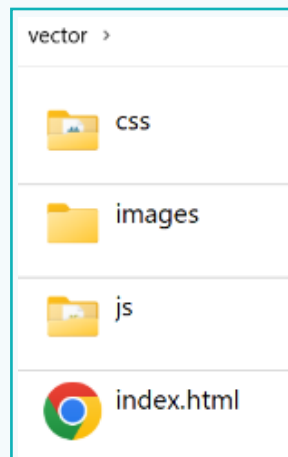
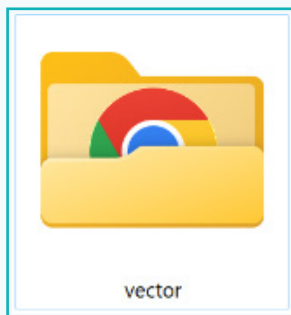
These are just a few examples of the diverse and ever-expanding applications of autonomous agents. To create autonomous agents in p5.js, we will implement concepts such as object-oriented programming, vectors, and behavioural algorithms. In this lesson, we will begin by creating a mover agent that is based on the laws of physics and can also be controlled by the user. This will serve as the building block for creating fully autonomous agents.

Activity

We will start by creating a class for the moving of an object or an agent that can exhibit all the necessary properties and methods. We will create the activity using an offline code editor called VS Code.

Initial File Setup

- 1 Create a project folder with a suitable name and add necessary project files like index.html, style.css, p5.js, p5.sound.js, and sketch.js with the project linked to the p5js libraries as done in the previous lesson.



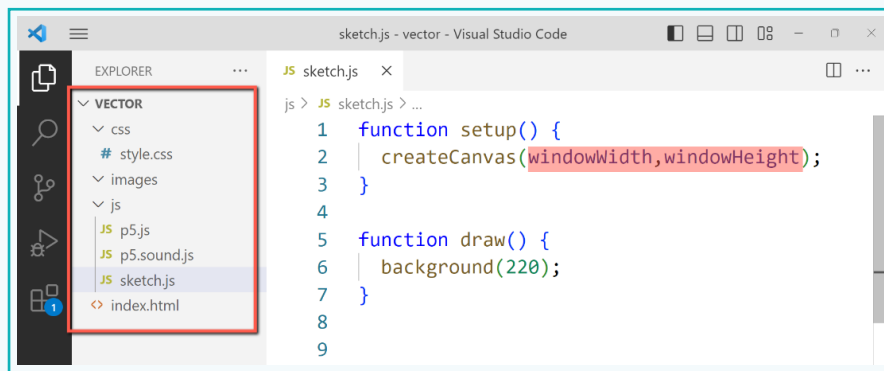
Try to recall and do this on your own. You can refer to previous lesson.

If required, use the QR code to download the project folder with basic files to start with a project.



Activity

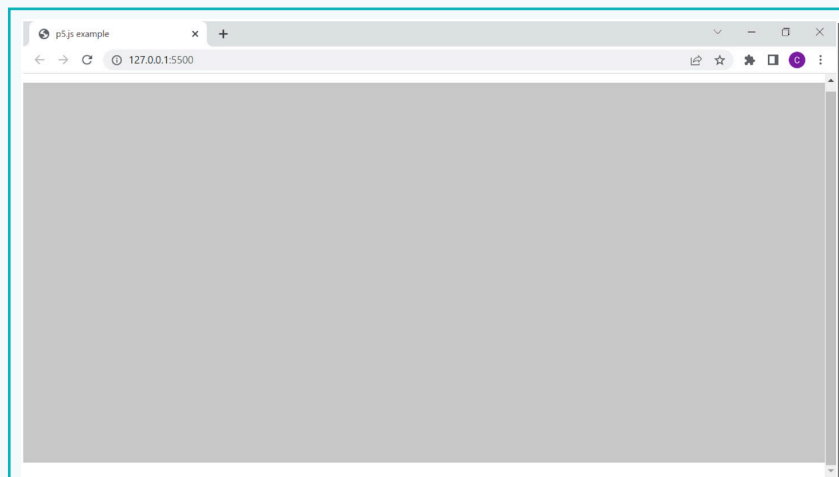
- 2 Open VS Code and open the project folder in the editor. The project files in the folder can be seen in Explorer on the left-hand side.
- Update the default code so that the canvas size will be the same as the size of the browser window using the default variables 'windowWidth' and 'windowHeight'.



Note:

'windowWidth' and 'windowHeight' are the default variables in p5js that store the width and height values of the inner window of the browser.

- 3 Click on the 'Go Live' option at the bottom bar or press 'Alt+L+O' to run the code and check the output in the browser.

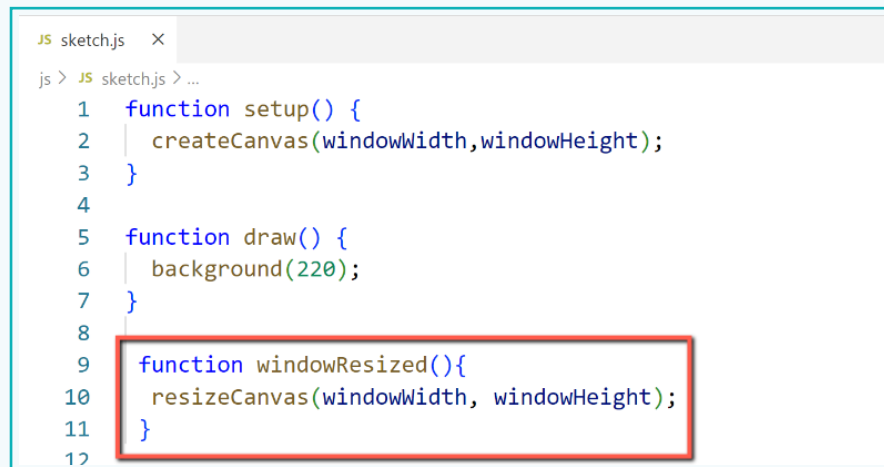


Activity

We can see the grey-coloured canvas all over the browser window. Change the canvas size to experiment with different looks. If we resize the browser window, the canvas size will remain constant. This can be addressed using a default function called 'windowResized()'.

windowResized() function is called once every time the browser window is resized. We can add the 'resizeCanvas()' command inside the function to update the size of the canvas same as the 'windowSize'.

- Add the function 'windowResized()' in the code.
- Add the 'resizeCanvas()' command inside the function to resize it to 'windowWidth' and 'windowHeight'.



```
JS sketch.js x
js > JS sketch.js > ...
1  function setup() {
2    createCanvas(windowWidth, windowHeight);
3  }
4
5  function draw() {
6    background(220);
7  }
8
9  function windowResized(){
10    resizeCanvas(windowWidth, windowHeight);
11  }
12
```

Exercise

Fill in the Blanks.

- _____ and _____ are the default variables that store the width and height values of the inner window of the GUI or browser.
- _____ is the default function in p5js that is executed whenever the browser window size is changed.
- Canvas size can be updated using the _____ command.

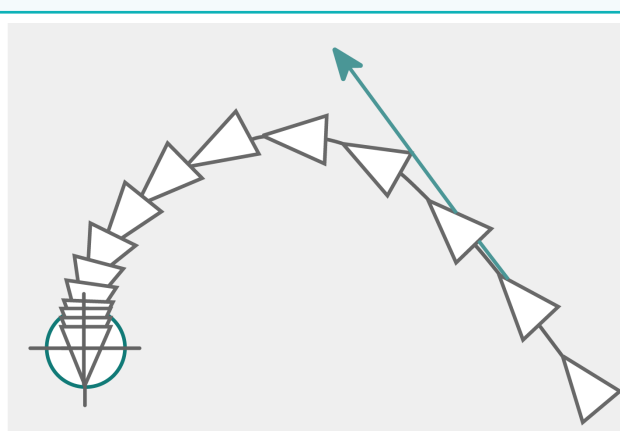
Activity

Now, we will implement OOP concepts to create an agent. Here is a general approach to creating Autonomous Agents:

- Define the Agent Class** that represents the autonomous agent containing properties and methods. An agent's properties can include its position, velocity, acceleration, and other relevant attributes, whereas the methods can include displaying or updating the agent's state and performing actions.
- Create an Object** from the agent's class and call class methods to render the object.
- Define Behaviours** and **Implement methods** within the class to update position, velocity, and acceleration.
- Interaction** of the agent with the environment can be added with user input to control the agent's behaviour.
- Implement Steering Algorithms** such as seek, flee, arrive, etc. to control the agent's movement and make it autonomous to appear intelligent.



Path Following



Arrive Behaviour

All the algorithms are based on finding and implementing the combined forces that influence the agent's acceleration and velocity as required for a 'steering behaviour'. Hence, implementation details depend on the application or simulation we want to create. The above steps give a general approach toward such applications and may vary slightly to suit the desired outcome in the p5.js framework. Let us begin the activity by creating a class named Mover using given steps.

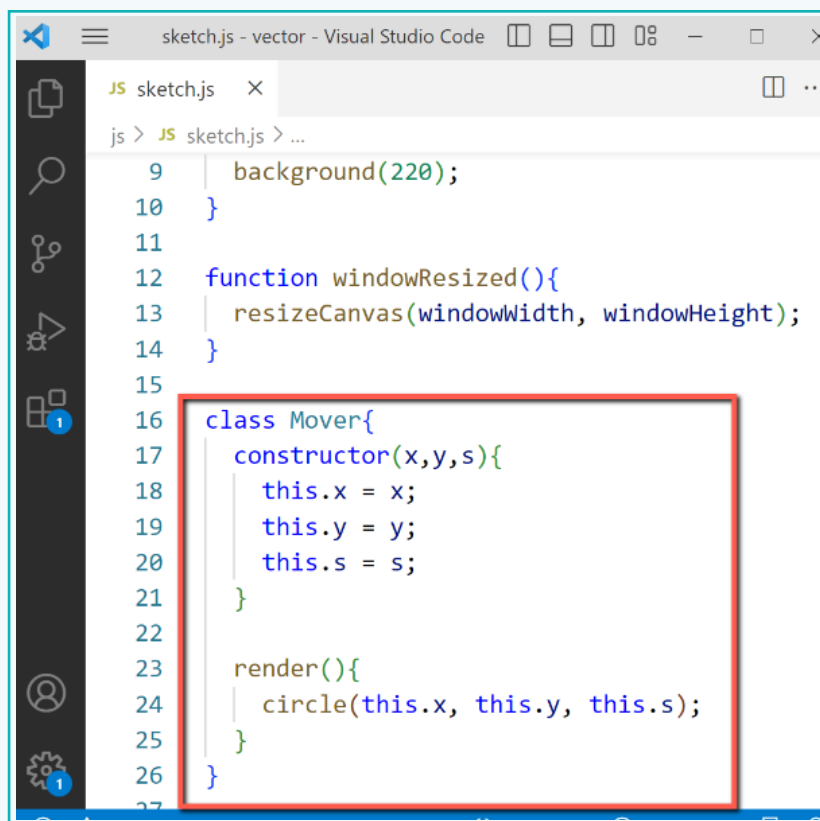
Activity

Mover Class

Let us create the class for the mover agent with its base properties and a 'render' method.

1 Create a Mover class

- Create a class 'Mover' and add the default 'constructor' method.
- Add input arguments 'x' and 'y' for x and y positions. Also, add input argument 's' as base dimension of the agent for defining size.
- Add parameters 'this.x', 'this.y', and 'this.s' in the 'constructor' method as 'class properties' and assign input attributes 'x', 'y', and 's', respectively.
- Add another method 'render()' in the class and add a 'circle()' command at position 'this.x' and 'this.y' of size 'this.s'.



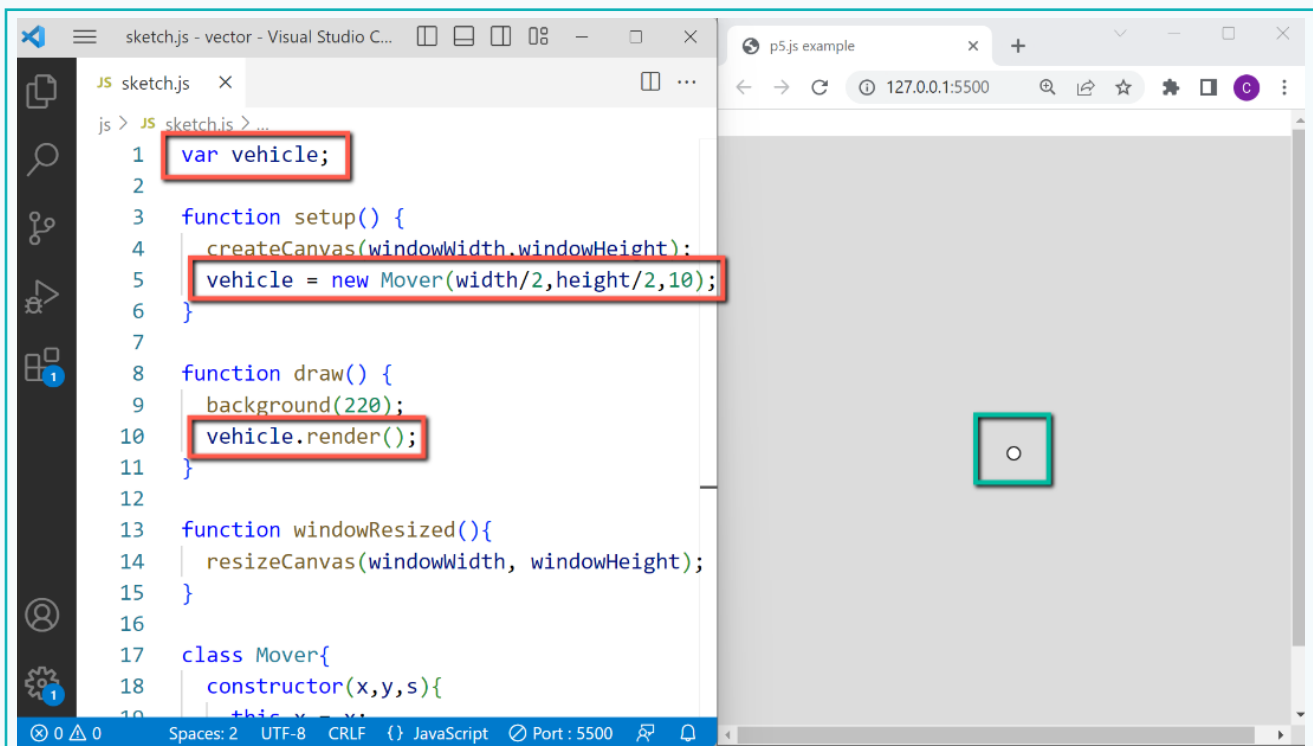
```
9 | background(220);
10 | }
11 |
12 | function windowResized(){
13 |   resizeCanvas(windowWidth, windowHeight);
14 | }
15 |
16 | class Mover{
17 |   constructor(x,y,s){
18 |     this.x = x;
19 |     this.y = y;
20 |     this.s = s;
21 |   }
22 |
23 |   render(){
24 |     circle(this.x, this.y, this.s);
25 |   }
26 | }
```

To check the working of the code, we can create an object and call the 'render' method to see the output.

Activity

2 Check the output

- Create a variable called 'vehicle' at the beginning of the code.
- Create an object of the 'Mover' class in setup() using the 'new' keyword with inputs for 'x', 'y', and 's' as 'width/2', 'height/2', and '10', respectively. Assign the object to the 'vehicle' variable.
- Call the render method for the 'vehicle' object in the 'draw' function. Run the code to display the object on the canvas.



We will be able to see a small circle of diameter '10 pixels' at the centre of the output screen. Now, let us add basic commands to create a general code structure of object positioning, movement, and rotation in the canvas.

Activity

3 Add position and movement

- Encapsulate the 'circle()' command in the render method by using the 'push()' and 'pop()' commands to maintain the translation, movements, and rotations associated with this object only.
- Add the 'translate(this.x, this.y)' command before circle() and change the position of the object to (0,0) to move the origin of the reference frame to the same position of the object. Now, the centre of the rotation will be the same as the centre of the object.

```

js > JS sketch.js > ...
13 function windowResized(){
14   resizeCanvas(windowWidth, windowHeight);
15 }
16
17 class Mover{
18   constructor(x,y,s){
19     this.x = x;
20     this.y = y;
21     this.s = s;
22   }
23
24   render(){
25     push()
26     translate(this.x,this.y);
27     circle(0,0, this.s);
28     pop();
29   }
30 }
  
```

With this basic setup, we can now add code for motion dynamics by implementing the laws of physics. We have already learnt concepts such as speed, velocity, acceleration, and other terms associated with the laws of motion in physics. Let us recall these concepts before we implement them in our code.

Exercise

Answer the questions with one sentence

- 4 Which default method is used in the class definition to initialise the class properties?



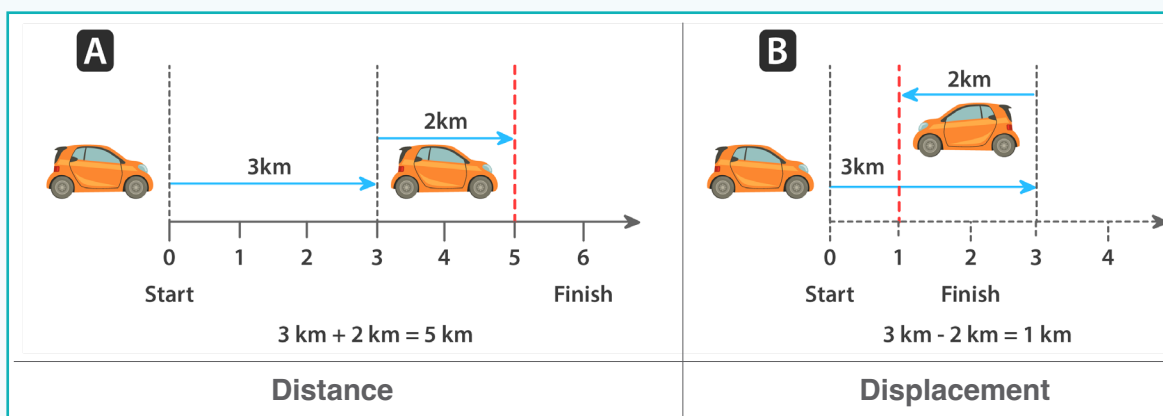
- 5 Which commands are used to limit the effect of translation and rotation effects to the only desired shape or entity on the canvas?



Activity

Scalar vs Vector

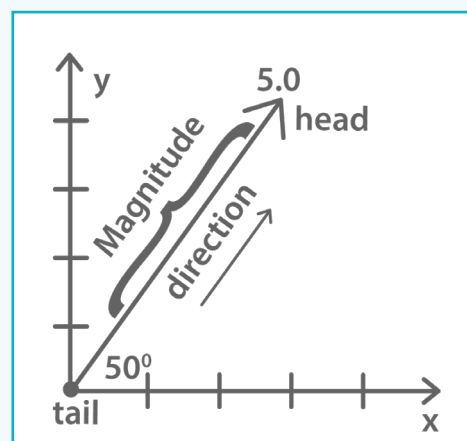
We measure how fast an object is moving using a term called speed or velocity. We know that **speed** is a physical quantity that only has magnitude. Such physical entities are called **Scalars**. However, velocity has magnitude and direction. Such physical entities are called **Vectors**. Another example is Distance vs Displacement. Distance is a scalar quantity, whereas Displacement is a vector quantity.



Activity

In mathematics and physics, vectors are used to describe various quantities in different contexts, such as position, velocity, force, acceleration, and more, in a particular coordinate system

A **vector** is a physical entity that has both a magnitude and a direction. It is represented using a line segment with an arrowhead pointing in the direction of the vector and whose length is the magnitude of the vector. The direction of the vector is from its tail to its head.

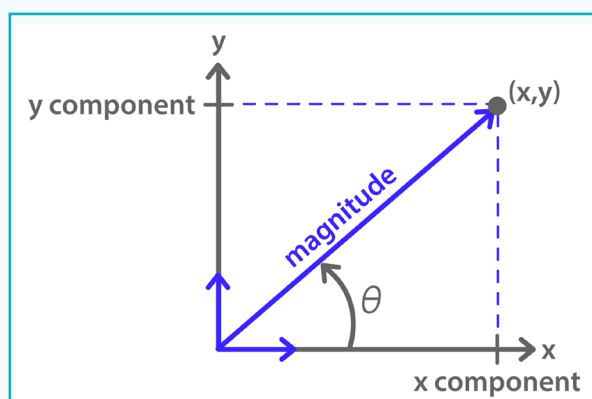


Here are the key characteristics of vectors –

- a. Magnitude:** A vector's 'magnitude' represents its size or length. It is typically represented by a positive scalar value. For example, a vector with a magnitude of '5' represents a quantity of size '5'.
- b. Direction:** A vector's 'direction' specifies the orientation or angle to which it points. It is often represented by an angle relative to a reference direction or by specifying its components in the coordinate system.

Vector Components

A vector can be broken down into its components along different axes of coordinate systems. In two-dimensional space, a vector can be represented by its x and y components. For example, x and y coordinates of a point are the x and y components of the 'position' vector.



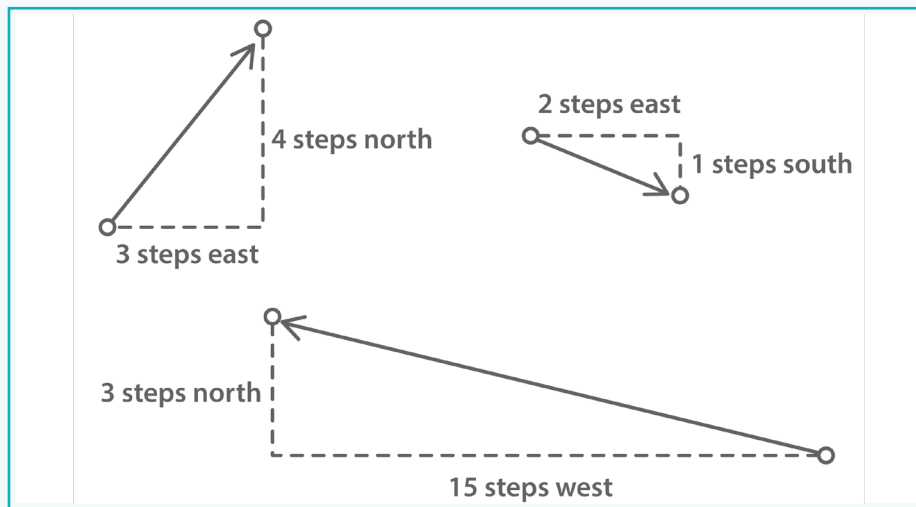
Note:

Note that in conventional coordinate systems, the positive Y-axis points upward, as illustrated in the images above. However, in the p5.js coordinate system, the positive Y-axis points in the opposite direction, i.e., downward. We will use the p5.js coordinate system in the next example.

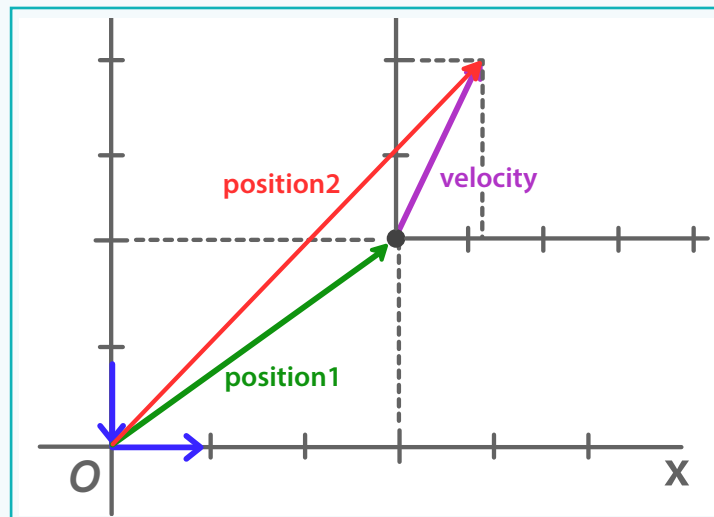
Activity

Relation between Vectors and Components

A vector can always be represented using two components along the x and y axes in a two-dimensional world. For example, '3 steps' in '+ve x' and then '4 steps up' towards '-ve y' (as per the p5js coordinate system), we can draw a vector from the start point to the endpoint.

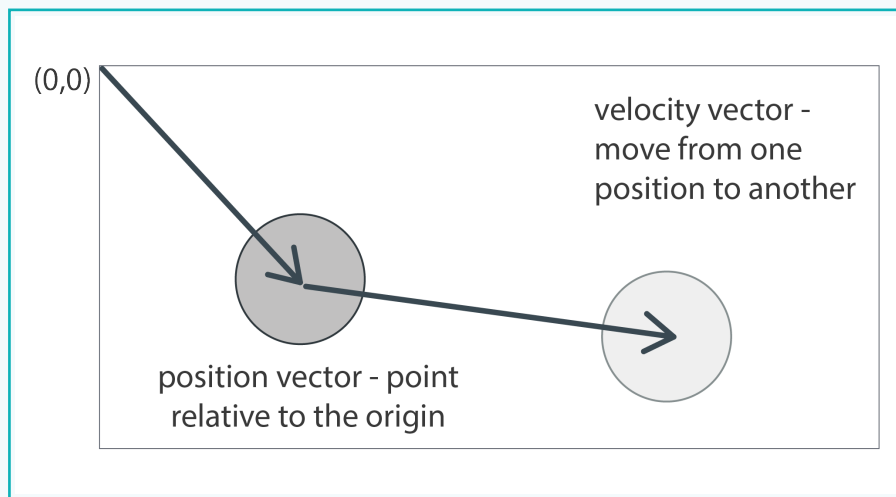


Similarly, we can move an object by a certain number of pixels horizontally and vertically to reach a specific location on the screen. We add the velocity components 'xVel' and 'yVel' to position components 'xPos' and 'yPos', respectively to move the agent from one position to another at a velocity. Hence, 'velocity' is the vector that represents the difference between two positions travelled in a unit time.



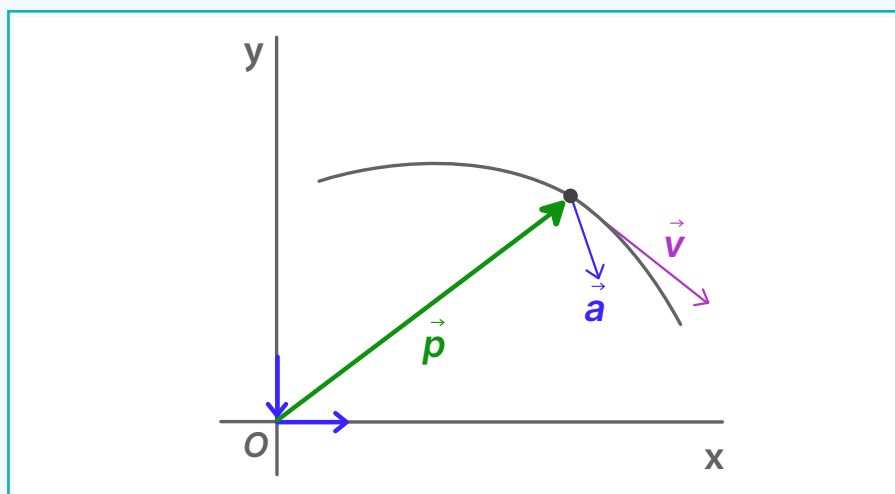
Activity

Just like velocity, Position is not a vector that describes a point in space. It is the path taken from the origin to that location in a 2D plane. In other words, a 'position' is a vector representing the difference between current location on the canvas and origin.



Vectors are widely used in many fields as they provide a powerful mathematical tool for representing and analysing quantities that have both magnitude and direction.

In p5.js, vectors are often used to represent positions in space, directions, velocities, acceleration and various other quantities. Vectors are used in computer graphics, physics simulations, and game development in two or three-dimensional space. To work with vectors in p5.js, we can use the built-in p5.Vector class, which provides a set of methods for creating vector quantities and easily manipulating them.



Activity

We can represent each point using the x and y coordinates. Also, each property that is vector, i.e., has direction such as position, velocity, or acceleration is represented using the x and y components.

For example components of the following quantities can be,

Position	xPos and yPos
Velocity	xVel and yVel
Acceleration	xAcc and yAcc

We have to use two variables for the x and y components of each property. For large applications, using two variables to represent each physical entity can make the code very complex. Also, if we create a project with 3D output, we will have to create three variables for each entity along the x, y, and z axes. Instead, we can use p5js vectors which encapsulate all components in one entity.

Creating a Vector

In p5js, we can create a vector by calling the 'createVector()' command and passing the initial x, y, (and optionally, z) coordinates as arguments directly instead of creating variables for the x and y components separately of each physical entity.

For example, consider the component variables as:

```
var xPos = 200;  
var yPos = 200;  
var xVel = 2.5;  
var yVel = 1.2;
```

In p5js, we can simply create Vectors as:

```
var position = createVector(200,200);  
var velocity = createVector(2.5,1.2);
```

Individual components of the vectors can be accessed by appending the '.x' and '.y' operators to the vector. Therefore, 'position.x' will have a value of '200' or 'velocity.y' will have a value of '1.2'.

Activity

Implement Vectors

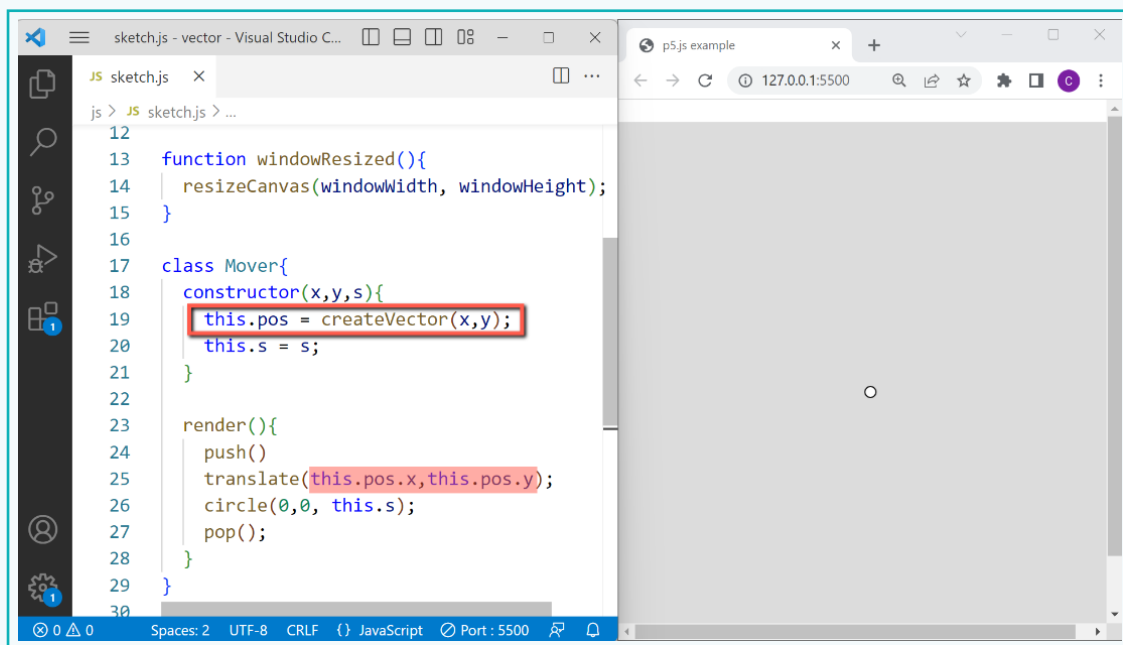
To implement vectors in the activity, we need to update the code to replace the variables with vectors. In our case, we created the 'property' variables in the class constructor. Therefore, we do not need to use the 'var' keyword. Instead, we will prepend 'this.' operator to the vector 'pos'.

```
this.pos = createVector(x,y);
```

We used the x and y coordinates in the 'render' method for the 'translate' command. Now, we can replace them by the x and y components of the 'pos' vectors to set the position of the agent, i.e., 'this.pos.x' and 'this.pos.y'.

Note that each property and method of the class that we use in the class definition must prepend 'this.' Keyword.

- 1 Remove the code added for creating 'this.x' and 'this.y' property variables in the 'constructor' method and instead, create a 'pos' vector.
- 2 Update the input values for the 'translate' command in the 'render()' method by the 'pos' vector components.



We can see that, after using vectors, code is much more simplified.

Exercise

State if the statements are True or False

6 To create a 2D vector using the x and y component values, the 'createVector()' command is used.

7 If a 'property' vector named 'this.velocity' is created in a 'Mover' class, its x and y components can be accessed as 'this.x' and 'this.y'.

Activity

Moving Object

To move the object, we will have to add velocity to the 'Mover' object. **Velocity** is a vector quantity that describes the rate of change of an object's displacement with respect to time.

$$\text{velocity} = (\text{position2} - \text{position1}) / \text{time required to move}$$

In the code, we consider framerate, i.e., time required to render each frame as the unit time. Hence, this is the time taken to displace the object at the second frame position from the first one. Hence,

$$\text{velocity} = (\text{position2} - \text{position1}) / 1$$

$$\text{velocity} = \text{position2} - \text{position1}$$

Hence,

$$\text{position2} = \text{position1} + \text{velocity}$$

We will create a velocity vector and add velocity to the position to update the object to its new position.

For operations like the addition of p5js vectors, we cannot use the usual operators like '+'. Instead, we have the 'vector operation' method.

Activity

Vector Operations

Vectors can undergo various operations, such as addition, subtraction, scaling, and more. These operations allow for mathematical manipulation and calculations involving vectors. Here are some methods to perform various vector operations.

add()	Add vectors
sub()	Subtract vectors
mult()	Scale the vector magnitude with multiplication
div()	Scale the vector magnitude with division
limit()	Limit the magnitude of a vector
heading()	The 2D heading of a vector expressed as an angle
setHeading()	Rotate the vector to a specific angle with the same magnitude
random2D()	Make a random 2D vector

Without vectors, we add each x and y components of position and velocity separately.

```
this.xPos = this.xPos + this.xVel;
```

```
this.yPos = this.yPos + this.yVel;
```

However, we can simply add vectors using the '.add()' method to add vectors.

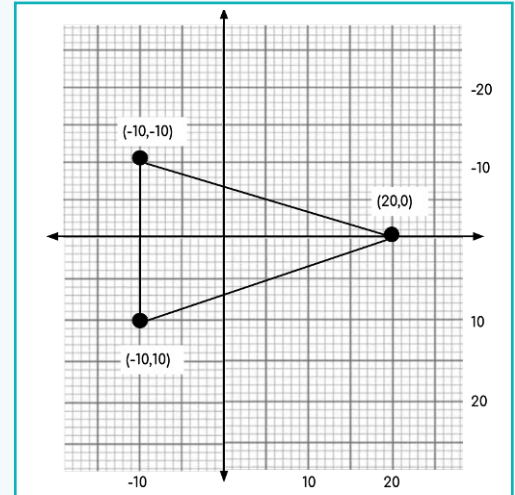
```
this.pos.add(this.vel);
```

Note:

We cannot add vector objects using the simple addition operator '+' like 'pos = pos + vel;'. This is because the addition operator (+) is used for single variable values that store the numbers or strings.

Activity

Now that we see how `add()` is written, we can update our code using velocity. We will also change the shape of the agent to a triangle pointing towards the direction of movement.



2 Add velocity

- Create a new vector property `this.vel` in the `'constructor'` method with x and y components as `'1'` and `'0'`, respectively.
- Add a new class method `'update()'` and add code to update `'this.pos'` vector by adding `'this.vel'` vector.
- Update the `'render()'` method to change the shape of the agent from a circle to a triangle pointing towards the direction of the motion

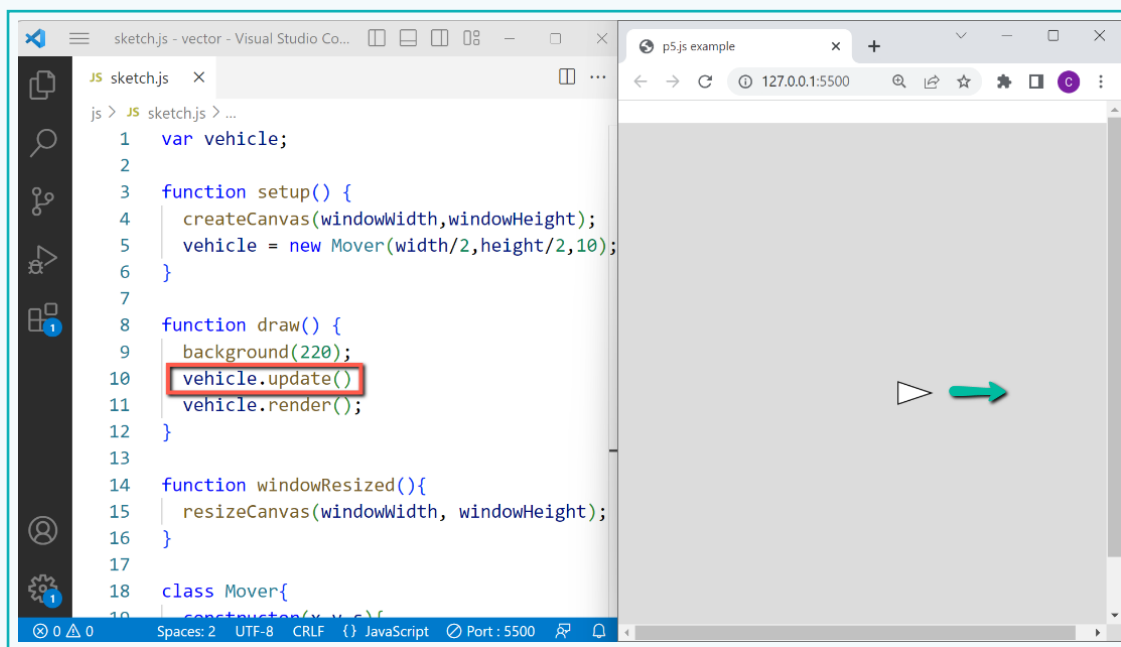
```

18 class Mover{
19   constructor(x,y,s){
20     this.pos = createVector(x,y);
21     this.vel = createVector(1,0);
22     this.s = s;
23   }
24
25   render(){
26     push()
27     translate(this.pos.x,this.pos.y);
28     triangle(-this.s,this.s,-this.s,-this.s,2*this.s,0);
29     pop();
30   }
31
32   update(){
33     this.pos.add(this.vel);
34   }
35 }
    
```

Activity

Note that there will be no change in the output as we have just created update method. However, we did not call that method for the object we created and displayed on the canvas.

- 3 Call the 'update()' method for the 'vehicle' object in the 'draw()' function before the 'render()' method.

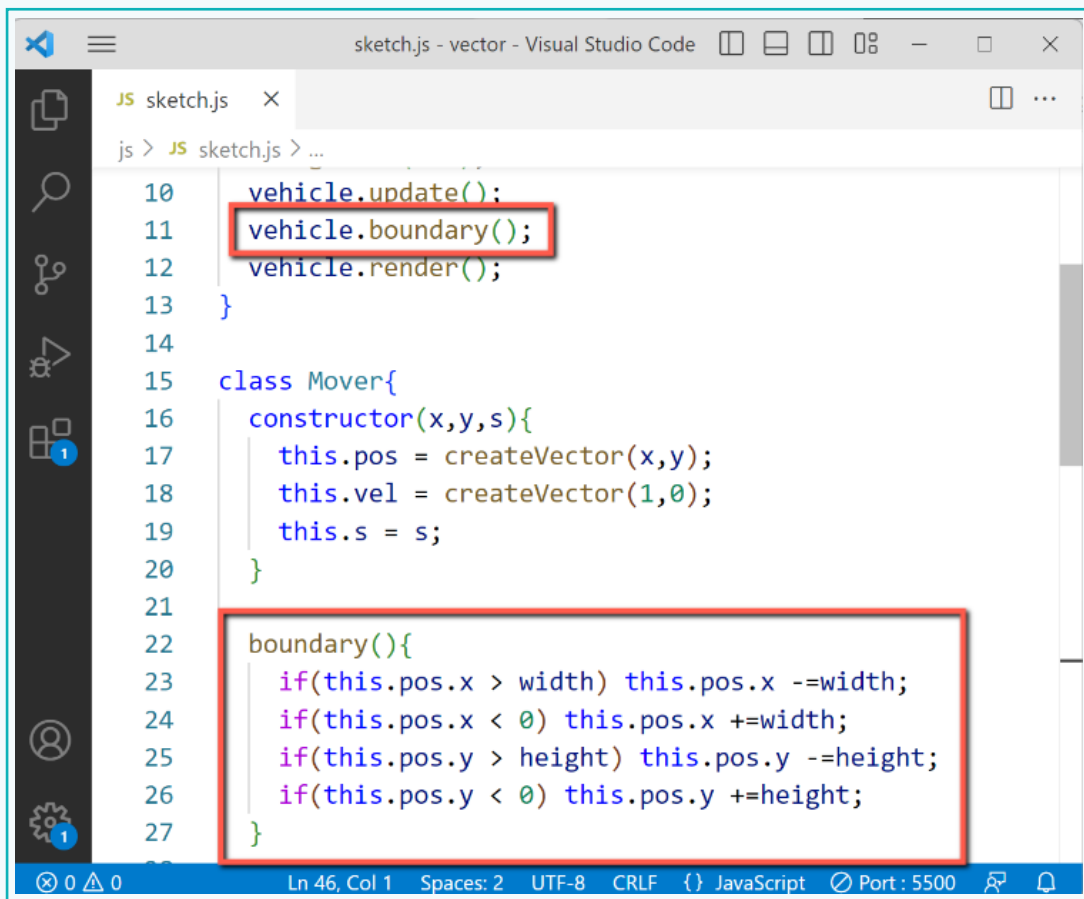


We will be able to see the agent moving in the forward direction with a constant velocity of '1' in the '+X' direction. Try different values of the x and y components in the 'velocity' vector and observe the movement speed and direction of the moving agent. As the object moves forward, it goes out of the screen. Let us remedy that by adding code to keep it on the canvas.

Activity

4 Add Boundary

- Create a new method boundary in the class 'Mover'.
- Add code using the 'if' condition such that if the object leaves one edge, it returns to the canvas from the opposite edge.
- Call the boundary method for the 'vehicle' class in the 'draw()' function.



```
10  vehicle.update();
11  vehicle.boundary();
12  vehicle.render();
13  }
14
15  class Mover{
16  constructor(x,y,s){
17    this.pos = createVector(x,y);
18    this.vel = createVector(1,0);
19    this.s = s;
20  }
21
22  boundary(){
23    if(this.pos.x > width) this.pos.x -=width;
24    if(this.pos.x < 0) this.pos.x +=width;
25    if(this.pos.y > height) this.pos.y -=height;
26    if(this.pos.y < 0) this.pos.y +=height;
27  }
```

Now, the object comes from the left edge as it moves out of the right. This will keep it inside the canvas.

Exercise

Match the pairs

- | | |
|-------------------|--|
| 8 a. add() | e. Limit the magnitude of a vector |
| b. mult() | f. Add vectors |
| c. limit() | g. Rotate vector to specific angle with the same magnitude |
| d. setHeading() | h. Scale the vector with multiplication |

Activity

Adding User Control

Now, we can move the object using velocity, let us add movement based on the user input. We can use the arrow keys to change the direction of the object. Also, we must update the velocity to a new vector in the direction of the angle changed by the user.

We have created a vector using its x and y components. We can also create a unit vector using its angle of direction. This can be created using a vector method **p5.Vector.fromAngle(angle)**.

Note:

A 'Unit' vector is a vector in a specific direction having a magnitude of '1'. 'Unit' vectors are mainly used to create or represent a vector in a specific direction.

Activity

5 Update direction with user input

- Create a new class property 'this.ang' and set it to '0' in the 'constructor' method to state the angle at which the vehicle is pointing.
- Create a 'control()' method in the class and add an 'if-else if' condition to check if the 'LEFT_ARROW' or the 'RIGHT_ARROW' key is pressed using the 'keyIsDown(keyCode)' command.
- If the right arrow is pressed, increment the 'this.ang' value by '0.05' to turn the agent in the right direction and if the left arrow key is pressed, decrement the 'this.ang' value by '0.05' to turn the agent in the left direction.
- Add the 'rotate(this.ang)' command in the 'render' method before the shape is drawn to turn the triangle in the direction changed by the user.
- Set the 'this.vel' vector to a unit vector in the direction of 'this.ang' using the 'p5.Vector.fromAngle()' command.

```

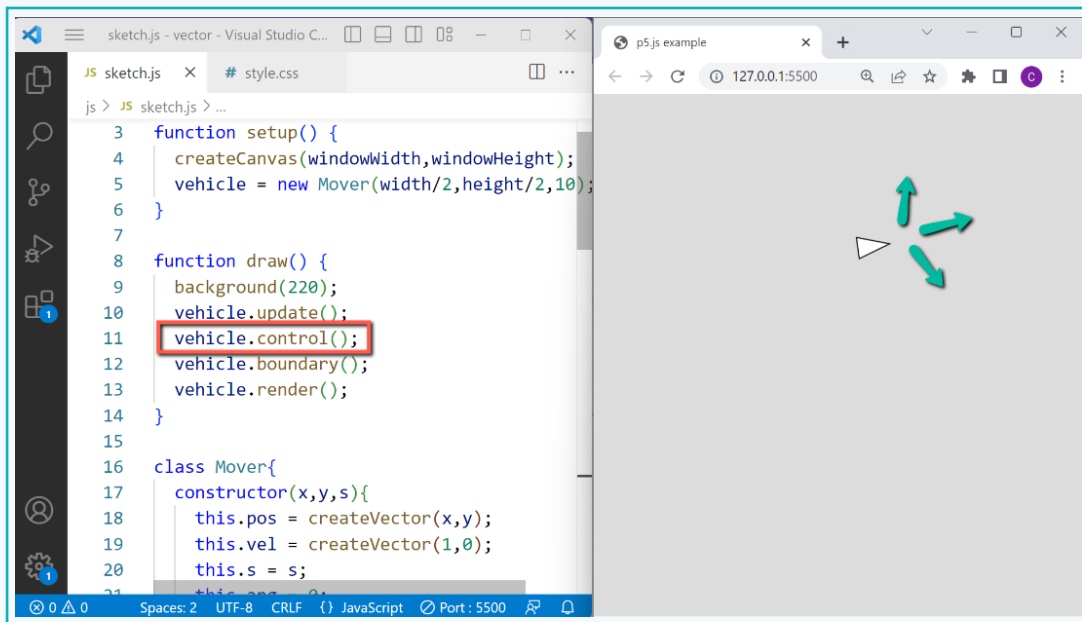
33   translate(this.pos.x, this.pos.y);
34   rotate(this.ang);
35   triangle(-this.s, this.s, -this.s, -this.s, 2*this.s, 0);
36   pop();
37 }
38
39 update(){
40   this.vel = p5.Vector.fromAngle(this.ang);
41   this.pos.add(this.vel);
42 }
43
44 control(){
45   if(keyIsDown(RIGHT_ARROW)){
46     this.ang += 0.05;
47   }else if(keyIsDown(LEFT_ARROW)){
48     this.ang -= 0.05;
49   }
50 }

```

Note that all angle values are in radians. Hence, we are incrementing or decrementing the angle by '0.05' radians, i.e. '2.86 ~ 3 degrees'.

Activity

- 6 Add the 'control' method for the 'vehicle' object in the 'draw()' function to implement the controls.
- Use the arrow keys to move the object in various directions at a unit velocity magnitude.

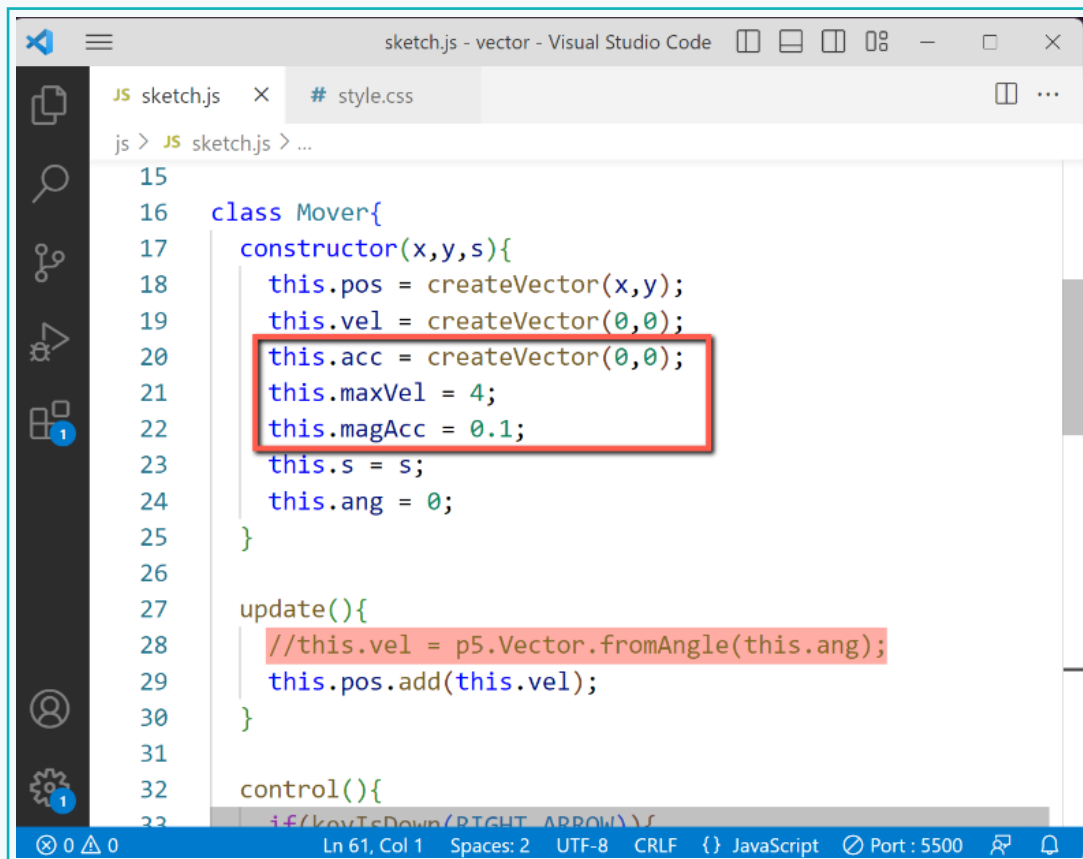


We are able to move the 'vehicle' object using the arrow keys, however it moves at a constant velocity magnitude. An object at rest changes its velocity to be able to start moving. This change in velocity is quantified as **Acceleration**. Let us add an 'acceleration' vector to the code to make the movement more realistic.

7 Add acceleration

- Create 'this.acc' property vector in the class 'constructor' using the 'createVector()' command with its initial x and y components set to '0'.
- Create 'this.maxVel' and 'this.magAcc' properties in the class 'constructor' to set the maximum velocity magnitude limit and acceleration magnitude to 4 and 0.1 respectively.
- Remove or comment out the code written earlier to set the value of 'this.vel' as we will update it later using the 'acceleration' vector.

Activity



```

15
16 class Mover{
17   constructor(x,y,s){
18     this.pos = createVector(x,y);
19     this.vel = createVector(0,0);
20     this.acc = createVector(0,0);
21     this.maxVel = 4;
22     this.magAcc = 0.1;
23     this.s = s;
24     this.ang = 0;
25   }
26
27   update(){
28     //this.vel = p5.Vector.fromAngle(this.ang);
29     this.pos.add(this.vel);
30   }
31
32   control(){
33     if(keyIsDown(RIGHT_ARROW)){

```

Acceleration is the rate of change of velocity per unit time.

Therefore,

Acceleration = (Velocity2 – Velocity1)/Time required to change the velocity

Consider the time required to update the next frame as unit time.

Acceleration = Velocity2 – Velocity1

Velocity2 = Acceleration + Velocity1

Hence, the updated velocity can be found by adding the 'acceleration' vector to the current velocity vector. We will add code such that when the up-arrow key is pressed, the vehicle will accelerate in the direction it is pointing and when the down-arrow key is pressed, it will decelerate or accelerate in the opposite direction. Let us implement this in the code.

Activity

8 User input for acceleration

- Add two 'if-else-if' code structures before the existing ones in the 'control' method to check if the 'UP_ARROW' and 'DOWN_ARROW' keys are pressed.
- If the up-arrow key is pressed, create a new unit vector pointing in the direction at 'this.ang' angle value using the 'p5.Vector.fromAngle(this.ang)' command.
- Use the 'mult()' vector command to set the vector magnitude to a scalar value 'this.magAcc'.
- In case of the down arrow, just set the magnitude of the 'this.acc' property to an opposite direction by multiplying 'this.magAcc' by '-1'.

```

33 control(){
34   if(keyIsDown(UP_ARROW)){
35     this.acc = p5.Vector.fromAngle(this.ang);
36     this.acc.mult(this.magAcc);
37   } else if(keyIsDown(DOWN_ARROW)){
38     this.acc = p5.Vector.fromAngle(this.ang);
39     this.acc.mult(-1*this.magAcc);
40   } else if(keyIsDown(RIGHT_ARROW)){
41     this.ang += 0.05;
42   } else if(keyIsDown(LEFT_ARROW)){
43     this.ang -= 0.05;
44   }
45 }
46
47 boundary(){
48   if(this.pos.x > width) this.pos.x -=width;
49   if(this.pos.x < 0) this.pos.x +=width;
50   if(this.pos.y > height) this.pos.y -=height;

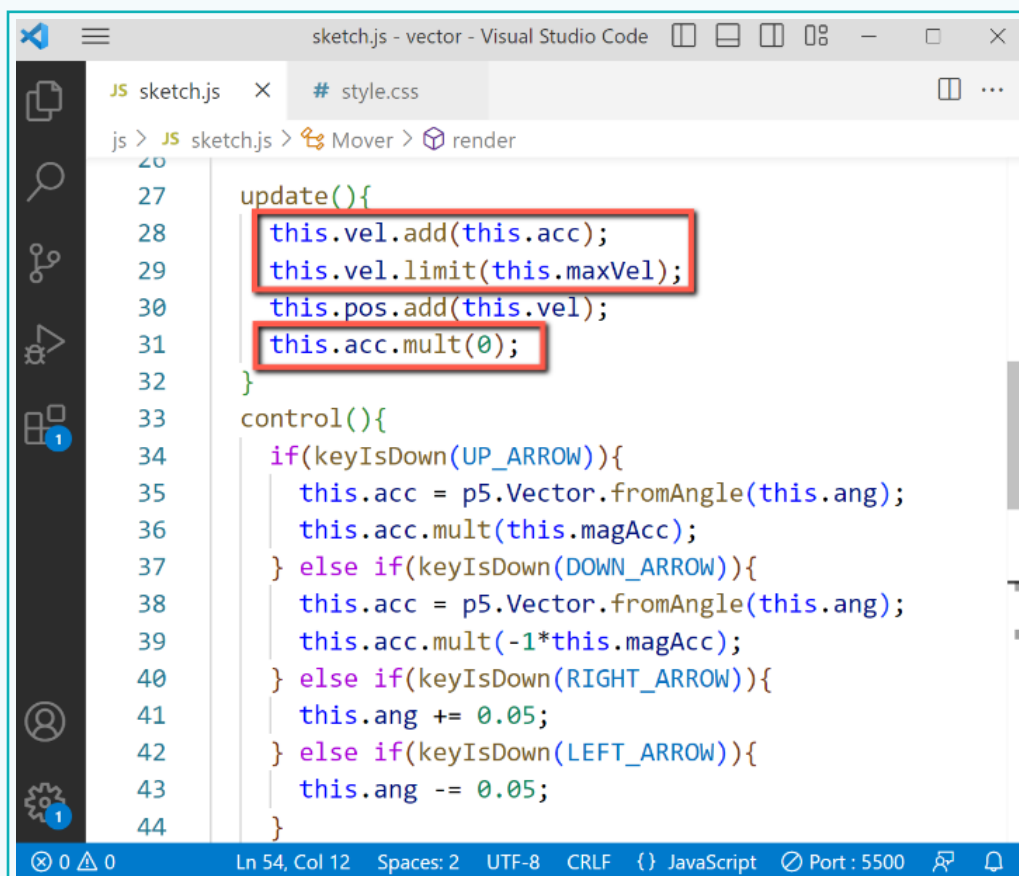
```

Now, we can set some acceleration and deceleration using the up and down arrow keys, but we have not yet used the 'acceleration' vector 'this.acc' to update the velocity and in turn, update the 'position' vectors. Once we do that, the vehicle will accelerate based on user input.

Activity

9 Update velocity and position with acceleration

- In the 'update()' method, add the 'this.acc' vector to the 'this.vel' vector at the beginning to update 'velocity' vector.
- Use the '.limit()' method to limit the velocity vector 'this.vel' magnitude to 'this.maxVel'.
- After the 'position' vector is updated, set the 'acceleration' vector 'this.acc' magnitude to '0' so that acceleration is only applied each cycle as long as the key is pressed, else there should be no acceleration.

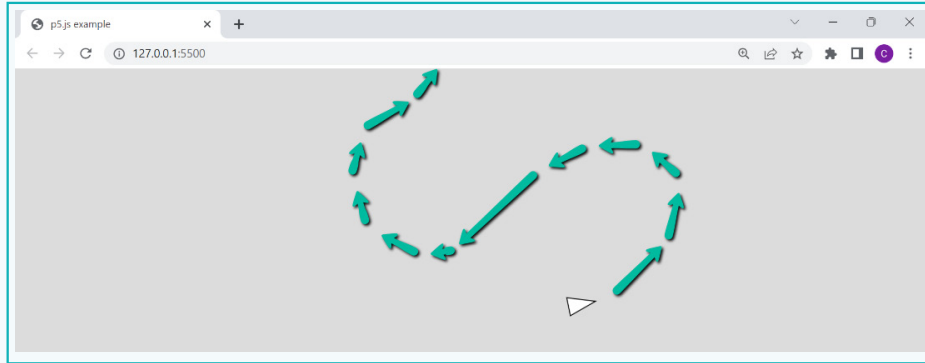


```

27  update(){
28      this.vel.add(this.acc);
29      this.vel.limit(this.maxVel);
30      this.pos.add(this.vel);
31      this.acc.mult(0);
32  }
33  control(){
34      if(keyIsDown(UP_ARROW)){
35          this.acc = p5.Vector.fromAngle(this.ang);
36          this.acc.mult(this.magAcc);
37      } else if(keyIsDown(DOWN_ARROW)){
38          this.acc = p5.Vector.fromAngle(this.ang);
39          this.acc.mult(-1*this.magAcc);
40      } else if(keyIsDown(RIGHT_ARROW)){
41          this.ang += 0.05;
42      } else if(keyIsDown(LEFT_ARROW)){
43          this.ang -= 0.05;
44      }
  
```

With this code implementation, we can see that the vehicle object can be moved and accelerated in different directions using the arrow keys. Note that the vehicle is moving freely in all directions without any loss due to friction. Also, even if the direction of the vehicle changes, the direction of movement does not change unless acceleration is applied in another direction. This simulates the motion of an object or a rocket in space, as there is no friction in a vacuum.

Activity



As we have used the 'if-else-if' condition structure, only one of the conditions can be true at a time. The code cannot detect multiple key press events. Therefore, if the vehicle is accelerating with the up-key pressed, even if the right or the left key is pressed to change the direction, it is not executed. This can be solved by simply changing the conditions to only simple four 'if' conditions.

10 Improve control

- Update the code in the 'control' method to have only the 'if' conditions instead of the 'if else' structure to get the motion effect of the multiple key presses.

```

35
36 control(){
37   if(keyIsDown(UP_ARROW)){
38     this.acc = p5.Vector.fromAngle(this.ang);
39     this.acc.mult(this.magAcc);
40   }
41   if(keyIsDown(DOWN_ARROW)){
42     this.acc = p5.Vector.fromAngle(this.ang);
43     this.acc.mult(-1*this.magAcc);
44   }
45   if(keyIsDown(RIGHT_ARROW)){
46     this.ang += 0.05;
47   }
48   if(keyIsDown(LEFT_ARROW)){
49     this.ang -= 0.05;
50   }
51 }
52

```

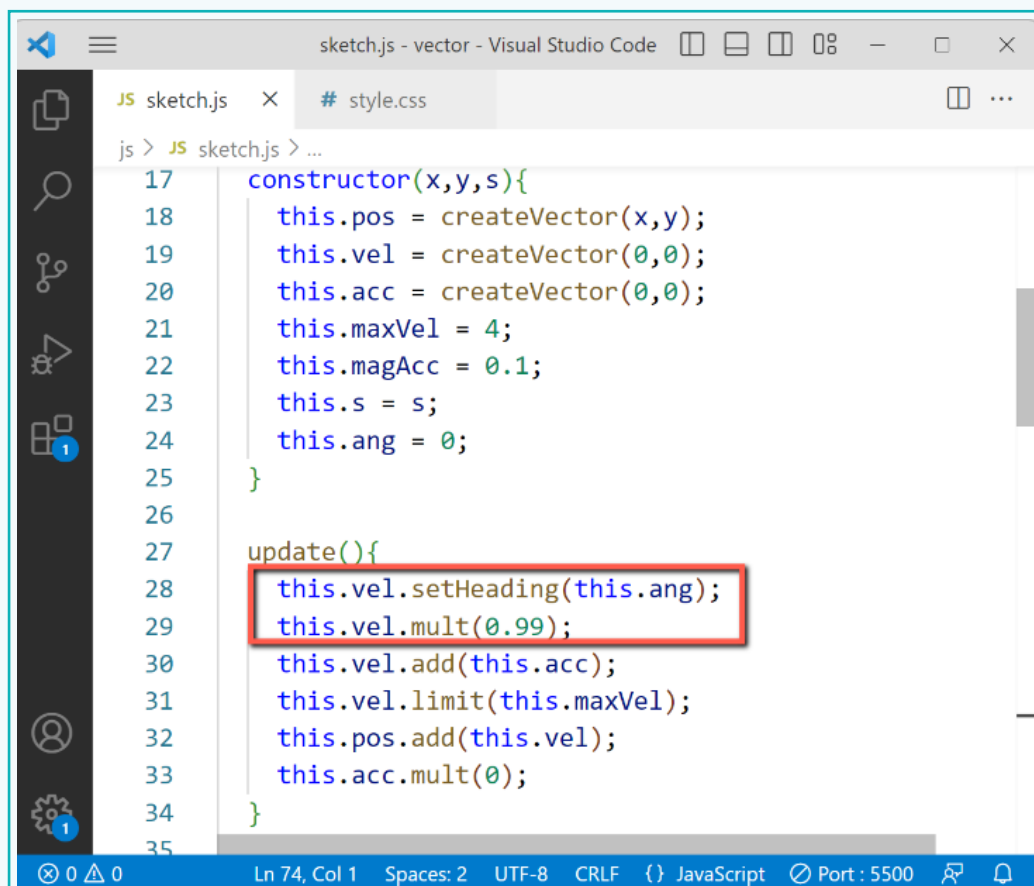
Activity

With this, we have implemented the laws of physics to simulate the motion of an object, which the user can control using the arrow keys. We can also modify this to simulate the motion of a vehicle on a surface, which includes friction between the tyres and the surface. Because of friction, two main things occur-

1. If not accelerated, the vehicle slows down eventually due to the friction on a plane road.
2. Also, if a turn is taken while running, the vehicle moves in that direction again due to friction.

Let us try to implement this phenomenon. We need to gradually reduce the magnitude of the 'velocity' vector. Also, as the direction changes, we need to change the velocity angle so that it aligns with the direction in which the vehicle is pointing. This can be achieved by using the 'setHeading(angle)' method, which changes the direction of a vector based on the input angle.

- a. Use the 'setHeading(this.ang)' method to 'this.vel' vector to update the direction of 'velocity' vector to the direction of the vehicle changed using the arrow keys.
- b. Also, multiply the 'this.vel' vector by a scalar value of '0.99' so that the magnitude keeps decreasing slowly.



```
js > JS sketch.js > ...
17 constructor(x,y,s){
18   this.pos = createVector(x,y);
19   this.vel = createVector(0,0);
20   this.acc = createVector(0,0);
21   this.maxVel = 4;
22   this.magAcc = 0.1;
23   this.s = s;
24   this.ang = 0;
25 }
26
27 update(){
28   this.vel.setHeading(this.ang);
29   this.vel.mult(0.99);
30   this.vel.add(this.acc);
31   this.vel.limit(this.maxVel);
32   this.pos.add(this.vel);
33   this.acc.mult(0);
34 }
35
```

Activity

Note:

Note if a number ' < 1 ' is multiplied by the same number repeatedly, its value keeps decreasing. For example, $0.9 \times 0.9 = 0.81$, $0.81 \times 0.9 = 0.729$ and so on.

Now, we can see that the vehicle slows down, eventually. Also, if we turn the vehicle, it moves in the changed direction, even if the acceleration is not applied. With this, we have completed our activity.

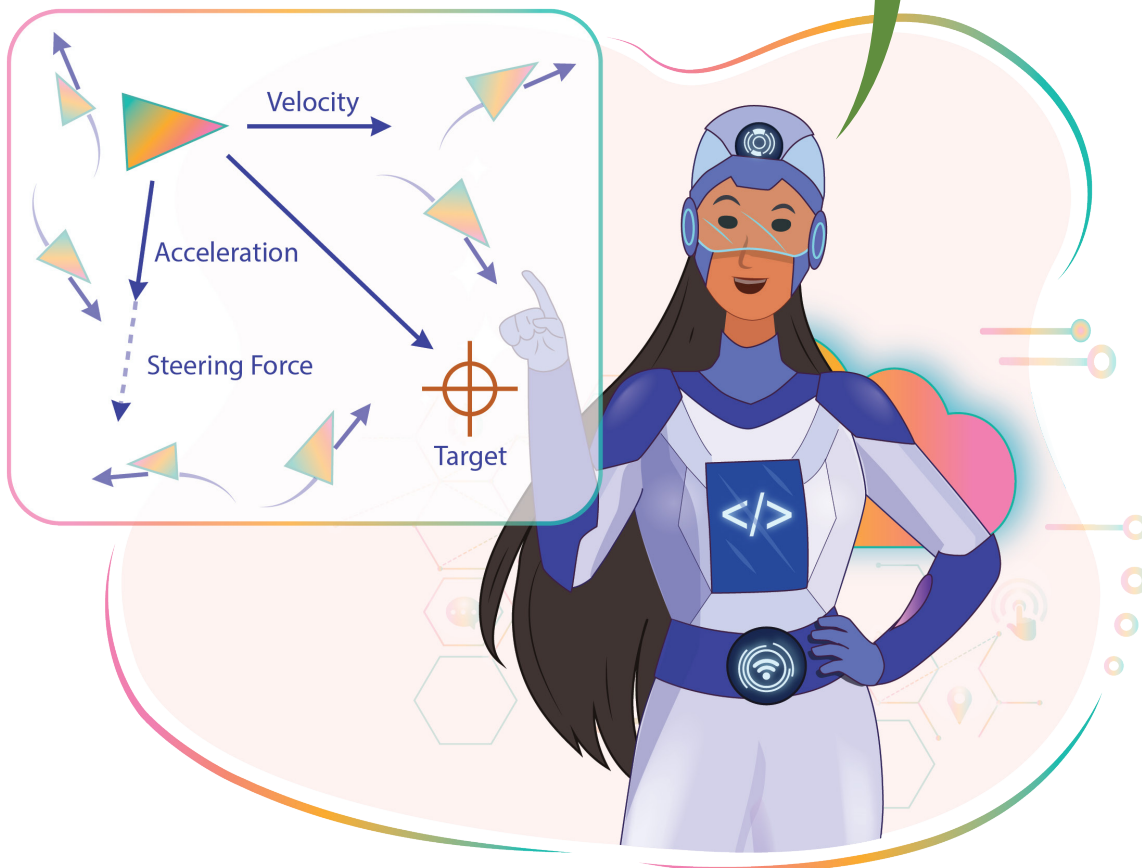
In this lesson, though we have not created a completely Autonomous agent, it is an important building step that includes the implementation of vectors and laws of physics to simulate vehicle movement. We can build on these concepts to create a completely autonomous agent of a specific steering behaviour.

Exercise

 Tick mark ☒ the correct answer

- 9** The _____ command is used to create a vector from a given angle.
- | | | | |
|--------------------------------|--------------------------|---------------------------------------|--------------------------|
| a. <code>createVector()</code> | ☐ | b. <code>mult()</code> | ☐ |
| c. <code>limit()</code> | ☐ | d. <code>p5.Vector.fromAngle()</code> | ☐ |
- 10** The 'Unit' vector is a vector with magnitude _____.
- | | | | |
|-------|--------------------------|------|--------------------------|
| a. 0 | ☐ | b. 1 | ☐ |
| c. 10 | ☐ | d. 2 | ☐ |
- 11** To change the direction of a vector to the opposite one, we can multiple the vector by _____.
- | | | | |
|-------|--------------------------|-----------------|--------------------------|
| a. 1 | ☐ | b. 0 | ☐ |
| c. -1 | ☐ | d. All of above | ☐ |

Movements in games and virtual worlds feel real because of the Physics algorithms used in Game engines developed by programmers. Modern games use engines like Unreal and Unity which are built on very complex algorithms. We have just taken a small step in that direction!



Tech Flash

- **Autonomous Agents** are virtual objects that possess a degree of autonomy in generating their own movements and actions.
- **Scalars** are the physical entities that only have magnitude.
- **Vectors** are physical entities that have magnitude as well as direction.
- **Components** of a vector are the projections of the vector along different axes of a coordinate systems.
- **Velocity** is rate of change of displacement per unit time.
- **Acceleration** is rate of change of velocity per unit time.